CO 2-Assessment Tool 1

NAME :B. Harsha Vardhan

SUB CODE: MLA0302

REG_NO: 192425201

EXP 1: Explain how Reinforcement Learning can be applied to optimize delivery routes in a logistics system. Identify the possible states, actions, rewards, and policy. Discuss how continuous learning improves efficiency over time.

CODE:

```
import random
# Possible locations (States)
locations = ["Warehouse", "A", "B", "C", "D"]
# Possible routes (Actions)
actions = ["Go_A", "Go_B", "Go_C", "Go_D"]
# Rewards for each destination
reward_table = {
    "Go_A": 10,
    "Go_B": 8,
    "Go_C": 12,
    "Go_D": 15
}
current_state = "Warehouse"
total_reward = 0
print("===== Logistics Delivery Route Optimization =====")
print("Starting Location:", current_state)
for episode in range(10):
   print("\nDelivery", episode + 1)
 action = random.choice(actions)
    print("Selected Route:", action)
 # Random traffic condition
    traffic = random.choice(["Low", "High"])
```

```
    if traffic == "Low":
        reward = reward_table[action]
 print("Traffic: Low")
        print("Delivery Successful")
    else:
        reward = reward_table[action] - 8
        print("Traffic: High")
        print("Delivery Delayed")
    total_reward += reward
    print("Reward:", reward)
    print("Total Reward:", total_reward)
print("\nSimulation Completed")
print("Final Total Reward:", total_reward)
```

OUTPUT :

```
Delivery 5
Selected Route: Go_A
Traffic: High
Delivery Delayed
Reward: 2
Total Reward: 23

Delivery 6
Selected Route: Go_D
Traffic: High
Delivery Delayed
Reward: 7
Total Reward: 30

Delivery 7
Selected Route: Go_B
Traffic: High
Delivery Delayed
Reward: 0
Total Reward: 30

Delivery 8
Selected Route: Go_C
Traffic: High
Delivery Delayed
Reward: 4
Total Reward: 34

Delivery 9
Selected Route: Go_D
Traffic: High
Delivery Delayed
Reward: 7
Total Reward: 41

Delivery 10
Selected Route: Go_D
Traffic: Low
Delivery Successful
Reward: 15
Total Reward: 56

Simulation Completed
Final Total Reward: 56
>>>
```

EXP 2 : Apply Reinforcement Learning concepts to a fraud detection system in fintech. Define the state space, action space, and reward mechanism. Explain how exploration strategies improve detection accuracy.

CODE :

```
import random
states = ["Low Amount", "High Amount", "International", "Known Customer", "Unknown Customer"]
actions = ["Approve", "Block"]
total_reward = 0
print("===== Reinforcement Learning Fraud Detection =====")
for i in range(10):
    print("\nTransaction", i+1)
    state = random.choice(states)
    action = random.choice(actions)
    actual = random.choice(["Fraud", "Legitimate"])
    print("State :", state)
    print("Action:", action)
    print("Actual:", actual)
    if actual == "Fraud" and action == "Block":
        reward, result = 10, "Correctly Blocked Fraud"
    elif actual == "Fraud" and action == "Approve":
        reward, result = -10, "Fraud Missed"
    elif action == "Approve":
        reward, result = 5, "Correctly Approved"
    else:
        reward, result = -5, "Legitimate Transaction Blocked"
    total_reward += reward
    print(result)
    print("Reward:", reward)
    print("Total Reward:", total_reward)
print("\n===== Simulation Completed =====")
```

print("Final Reward:", total_reward)

OUTPUT:

```
Transaction 6
State : Low Amount
Action: Block
Actual: Legitimate
Legitimate Transaction Blocked
Reward: -5
Total Reward: -5

Transaction 7
State : Known Customer
Action: Block
Actual: Legitimate
Legitimate Transaction Blocked
Reward: -5
Total Reward: -10

Transaction 8
State : High Amount
Action: Approve
Actual: Legitimate
Correctly Approved
Reward: 5
Total Reward: -5

Transaction 9
State : Known Customer
Action: Block
Actual: Fraud
Correctly Blocked Fraud
Reward: 10
Total Reward: 5

Transaction 10
State : Unknown Customer
Action: Approve
Actual: Legitimate
Correctly Approved
Reward: 5
Total Reward: 10

===== Simulation Completed =====
Final Reward: 10
```

EXP 3 : Illustrate how Reinforcement Learning is used in a streaming platform for content recommendation. Discuss the impact of delayed rewards, user feedback, and changing preferences on learning performance.

CODE :

```
import random
content = ["Action", "Comedy", "Drama", "Sci-Fi"]
rewards = {
    "Action": 10,
    "Comedy": 5,
    "Drama": 3,
    "Sci-Fi": 8
}
total_reward = 0
print("===== RL Content Recommendation System =====")
for i in range(5):
recommendation = random.choice(content)
    feedback = random.choice(["Watched", "Skipped"])
print("\nRecommendation:", recommendation)
    print("User Feedback:", feedback)
if feedback == "Watched":
        reward = rewards[recommendation]
    else:
        reward = -5
total_reward += reward
print("Reward:", reward)
    print("Total Reward:", total_reward)
print("\nLearning Completed")
print("Final Reward:", total_reward)
```

OUTPUT:

```
Recommendation: Drama
User Feedback: Skipped
Reward: -5
Total Reward: -5

Recommendation: Sci-Fi
User Feedback: Watched
Reward: 8
Total Reward: 3

Recommendation: Comedy
User Feedback: Watched
Reward: 5
Total Reward: 8

Recommendation: Action
User Feedback: Skipped
Reward: -5
Total Reward: 3

Recommendation: Action
User Feedback: Watched
Reward: 10
Total Reward: 13

Learning Completed
Final Reward: 13
```

EXP 4 : Evaluate the effectiveness of Reinforcement Learning in predictive maintenance compared to traditional rule-based approaches. Discuss risks and reliability concerns in industrial environments.

CODE:

```
import random

states = [
    "Normal Traffic",
    "Suspicious Traffic",
    "Attack Detected"
]

actions = [
    "Allow",
    "Monitor",
    "Block"
]

reward = 0
```

```
episodes = 10
print("Cybersecurity Intrusion Detection using Reinforcement Learning\n")
for episode in range(episodes):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
    if state == "Attack Detected" and action == "Block":
        r = 10
    elif state == "Suspicious Traffic" and action == "Monitor":
        r = 6
    elif state == "Normal Traffic" and action == "Allow":
        r = 5
    else:
        r = -5
    reward += r
    print("Reward:", r)
    print()
print("Training Completed")
print("Total Reward:", reward)
```

OUTPUT:

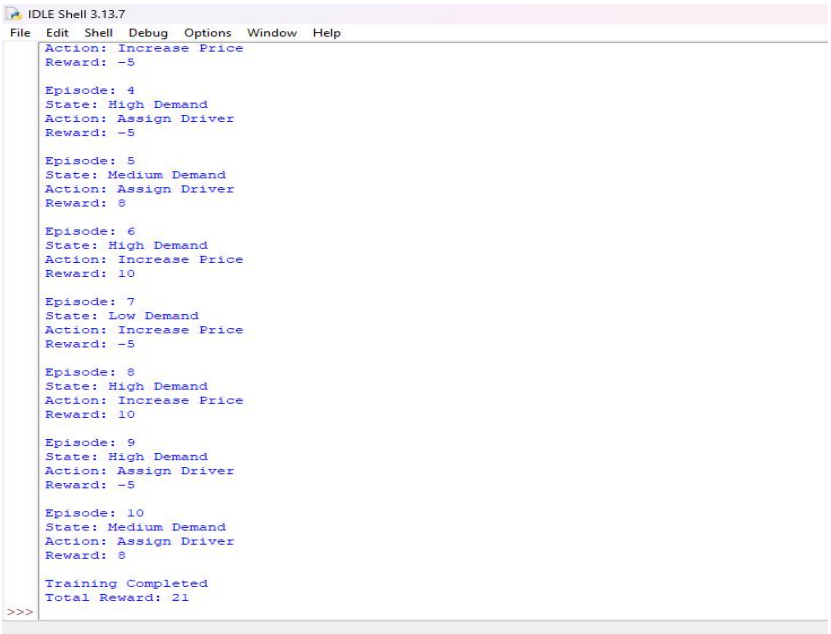
EXP 5: Analyze how Reinforcement Learning can be used to optimize transportation systems such as bus scheduling or route allocation. Define states, actions, and rewards, and discuss how real-time data improves performance.

CODE:

```
import random
states = [
    "Low Demand",
    "Medium Demand",
    "High Demand"
]
actions = [
    "Assign Driver",
    "Increase Price",
    "Keep Price"
]
total_reward = 0
episodes = 10
print("Ride Sharing using Reinforcement Learning\n")
for episode in range(episodes):
```

```
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
    if state == "High Demand" and action == "Increase Price":
        reward = 10
    elif state == "Medium Demand" and action == "Assign Driver":
        reward = 8
    elif state == "Low Demand" and action == "Keep Price":
        reward = 6
    else:
        reward = -5
    total_reward += reward
    print("Reward:", reward)
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT:



EXP 6: Apply Reinforcement Learning to dynamic pricing in e-commerce. Define the state space, action space, and reward function. Discuss challenges in balancing exploration and exploitation.

CODE:

```
import random

states = [
    "Low Demand",
    "Medium Demand",
    "High Demand"
]
actions = [
    "Decrease Price",
    "Keep Price",
    "Increase Price"
]
total_reward = 0
episodes = 10
print("Dynamic Pricing using Reinforcement Learning\n")
for episode in range(episodes):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
    if state == "High Demand" and action == "Increase Price":
        reward = 10
    elif state == "Medium Demand" and action == "Keep Price":
        reward = 8
    elif state == "Low Demand" and action == "Decrease Price":
        reward = 6
    else:
```

```
        reward = -5
    total_reward += reward
    print("Reward:", reward)
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT:

EXP 7: Illustrate the use of Reinforcement Learning in smart city waste management systems. Identify the MDP components and discuss how continuous updates improve operational efficiency.

CODE:

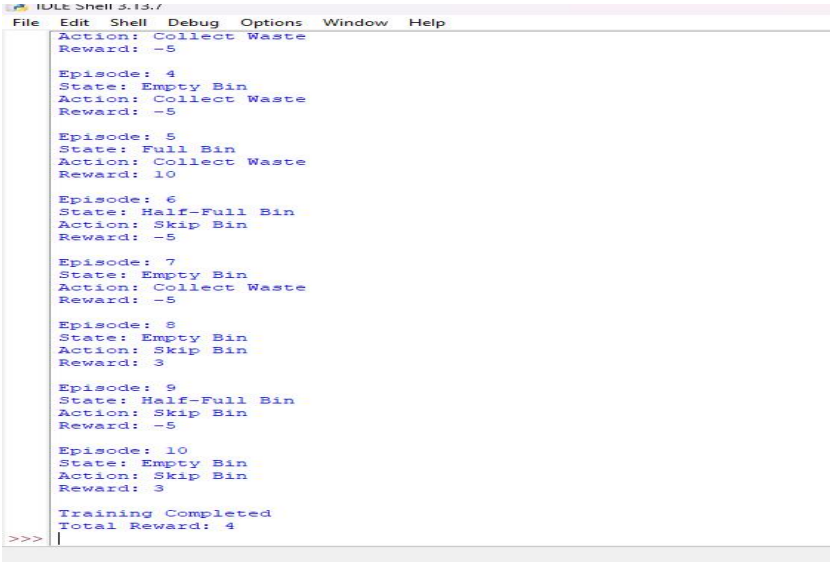
```
import random

states = [
```

```
    "Empty Bin",
    "Half-Full Bin",
    "Full Bin"
]
actions = [
    "Collect Waste",
    "Skip Bin"
]
total_reward = 0
episodes = 10
print("Smart City Waste Management using Reinforcement Learning\n")
for episode in range(episodes):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
    if state == "Full Bin" and action == "Collect Waste":
        reward = 10
    elif state == "Half-Full Bin" and action == "Collect Waste":
        reward = 5

    elif state == "Empty Bin" and action == "Skip Bin":
        reward = 3
    else:
        reward = -5
    total_reward += reward
    print("Reward:", reward)
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT:



EXP 8: Evaluate how Reinforcement Learning can be used in network bandwidth allocation in telecommunications. Define states, actions, and rewards, and examine how the system adapts to dynamic conditions.

CODE:

```
import random
states = [
    "Low Traffic",
    "Medium Traffic",
    "High Traffic"
]
actions = [
    "Allocate Low Bandwidth",
    "Allocate Medium Bandwidth",
    "Allocate High Bandwidth"
]
total_reward = 0
episodes = 10
print("Network Bandwidth Allocation using Reinforcement Learning\n")
for episode in range(episodes):
    state = random.choice(states)
    action = random.choice(actions)
```

```
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
    if state == "Low Traffic" and action == "Allocate Low Bandwidth":
        reward = 5
    elif state == "Medium Traffic" and action == "Allocate Medium Bandwidth":
        reward = 8
    elif state == "High Traffic" and action == "Allocate High Bandwidth":
        reward = 10
    else:
        reward = -5
    total_reward += reward
    print("Reward:", reward)
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT:

```
Episode: 5
State: Medium Traffic
Action: Allocate Low Bandwidth
Reward: -5

Episode: 6
State: Medium Traffic
Action: Allocate Medium Bandwidth
Reward: 8

Episode: 7
State: Low Traffic
Action: Allocate Low Bandwidth
Reward: 5

Episode: 8
State: High Traffic
Action: Allocate Low Bandwidth
Reward: -5

Episode: 9
State: Medium Traffic
Action: Allocate High Bandwidth
Reward: -5

Episode: 10
State: High Traffic
Action: Allocate High Bandwidth
Reward: 10

Training Completed
Total Reward: -12
```

EXP 9: Analyze the application of Reinforcement Learning in portfolio management in finance. Discuss advantages over traditional models and challenges such as risk and

delayed rewards.

CODE:

```
import random
states = [
    "Bull Market",
    "Stable Market",
    "Bear Market"
]
actions = [
    "Buy Stock",
    "Hold Stock",
    "Sell Stock"
]
total_reward = 0
episodes = 10
print("Portfolio Management using Reinforcement Learning\n")
for episode in range(episodes):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("Market State:", state)
    print("Action:", action)
    if state == "Bull Market" and action == "Buy Stock":
        reward = 10
    elif state == "Stable Market" and action == "Hold Stock":
        reward = 8
    elif state == "Bear Market" and action == "Sell Stock":
        reward = 6
    else:
        reward = -5
    total_reward += reward
    print("Reward:", reward)
```

```
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT:

EXP 10: Justify the use of Reinforcement Learning in renewable energy management

systems. Define the RL framework components and discuss the impact of environmental uncertainty on decision-making.

CODE:

```
import random
states = [
    "Low Renewable Energy",
    "Medium Renewable Energy",
    "High Renewable Energy"
]
actions = [
    "Store Energy",
    "Use Energy",
    "Buy Power from Grid"
]
total_reward = 0
episodes = 10
print("Renewable Energy Management using Reinforcement Learning\n")
for episode in range(episodes):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("Energy State:", state)
    print("Action:", action)
    if state == "High Renewable Energy" and action == "Store Energy":
        reward = 10
    elif state == "Medium Renewable Energy" and action == "Use Energy":
        reward = 8
    elif state == "Low Renewable Energy" and action == "Buy Power from Grid":
        reward = 6
    else:
        reward = -5
    total_reward += reward
```

```
    print("Reward:", reward)
    print()
print("Training Completed")
print("Total Reward:", total_reward)
```

OUTPUT: